

Downlink-Constrained Task Scheduling for Orbital Edge Computing

Hemanth Sudhakaran
NICE Lab, North Carolina State University

September 12, 2026

Abstract

This document works through the on-board compression and downlink scheduling problem for Orbital Edge Computing (OEC): a residual-quantized VAE (RQ-VAE) compresses each image at one of several selectable depths, and a rolling-horizon MPC scheduler decides depth, timing, and routing under time-varying satellite/ground-station connectivity. It covers the system model and full scheduling formulation – extending the original task model and MPC objective with a unified utility function, a congestion-aware routing variant, and hierarchical/two-level MPC couplings – and the experimental setup (constellation, link-capacity regimes, task workload) those schedulers are evaluated against. Results, related work, and a conclusion are being written up separately as the corresponding experiments are finished.

1 Introduction

A satellite in low Earth orbit sees far more of the ground than it can ever report. One pass over a wildfire or a flood can generate more raw pixels than the downlink can move to a ground station before the next decision has to be made, and a LEO satellite is only in radio contact with any given ground station for a few minutes before it drops below the horizon. Something has to decide which images matter, how hard to compress each one, and when to send it. That’s what Orbital Edge Computing (OEC) is: pushing that decision onto the satellite itself instead of shipping raw data down and sorting it out later.

This document works through that decision problem end to end. An on-board residual-quantized VAE (RQ-VAE) compresses each image at one of several depths (more refinement passes, bigger payload, better reconstruction), and a rolling-horizon MPC scheduler picks depth, timing, and route for every task competing for downlink capacity, under a topology and contact schedule that changes every slot.

§2 builds directly on the original system model and MPC formulation, writing out the full scheduling problem – the unified utility function, the concave coverage trick that keeps it a linear program, and every scheduler variant (flat MPC, congestion-aware routing, and the two ways of coupling a routing optimizer to the depth decision). §3 specifies the constellation, link-capacity regimes, and task workload that later experiments will run against. Results, related work, and a conclusion are still in progress and follow once the corresponding experiments are finished and written up.

2 System Model and Problem Formulation

This section states the system model and scheduling formulation this project implements. It builds directly on the original system-and-task model and rolling-horizon MPC objective (satellites $\mathcal{S}$, GBSs $\mathcal{G}$, tasks $\mathcal{K}$, depths $\mathcal{D}$, and the basic MPC structure), extending it with a concrete quality signal, a unified multi-term utility, and additional scheduler variants – congestion-aware routing, two-level and hierarchical MPC couplings, and an offline optimality bound – that later

sections evaluate. Every symbol below matches the implementation (`hypatia_sim/oec_sim/`) exactly; anywhere an earlier design intent and the shipped code differ, it's flagged inline instead of silently resolved.

2.1 Sets, time, and links

We consider a LEO orbital edge computing network consisting of a set of satellites $\mathcal{S}$ and a set of ground base stations (GBSs) $\mathcal{G}$. Time is divided into equal scheduling slots $t \in \mathcal{T}$, each of duration Δt . Because satellites move, the network topology is time-varying: let $\mathcal{E}_t$ denote the set of available inter-satellite links (ISLs) and satellite-to-ground links at slot t , where each available link $(i, j) \in \mathcal{E}_t$ has capacity $C_{ij}(t)$, obtainable from predicted contact windows and link states. In this paper, $\mathcal{S}$ is a 1,156-satellite Kuiper-630-shell Walker constellation, $\mathcal{G}$ is a 4-city ground segment, $\Delta t = 30$ s, and $\mathcal{E}_t$ is gated by a line-of-sight/range test for ISLs and an elevation test for ground links – full numeric values are in Table 1 (§3).

2.2 Tasks and compression depths

A set of image-transmission tasks $\mathcal{K}$ is generated when satellites observe fixed areas of interest. Each task $k \in \mathcal{K}$ is characterized by its source satellite s_k , destination GBS g_k , arrival time a_k , number of images N_k , importance weight w_k , and soft deadline d_k . Each task is compressed using one effective RQ-VAE depth selected from

$$\mathcal{D} = \{2, 4, 8, 16\}. \quad (1)$$

That's the whole choice a task makes: one of four compression strengths, trading payload size against how useful the reconstructed image ends up being.

For each depth $q \in \mathcal{D}$, let b_q denote the compressed payload size per image and t_q^{enc} the encoding delay. Rather than leaving downstream utility as an abstract placeholder u_q , we measure it: $b_q = 88q$ bytes (measured on the FLAIR-1 8×8 dedicated models), and quality is either the legacy pixel-fidelity proxy $u_q = 1 - \text{LPIPS}_q$ or the measured downstream segmentation quality s_q , both selectable via a configuration flag (which one is preferable, and why, is a results-section question for later):

$$s_q = \frac{\text{mIoU}_q - \text{mIoU}_{\text{floor}}}{\text{mIoU}_{\text{ref}} - \text{mIoU}_{\text{floor}}}, \quad (2)$$

where $\text{mIoU}_{\text{floor}}$ is the downstream segmentation model's score with the optical bands blanked out (near-infrared and elevation channels only) – the decision-theoretic zero for not delivering an image's optical content, rather than the mathematical floor $\text{mIoU} = 0$. So $s_q = 0$ means "as useless as not sending it," $s_q = 1$ means "as useful as the uncompressed original," and s_q in between reads directly as a fraction of the achievable task value recovered at depth q .

Let $x_{k,q} \in \{0, 1\}$ indicate whether task k is compressed using depth q . Each task selects exactly one compression depth:

$$\sum_{q \in \mathcal{D}} x_{k,q} = 1, \quad \forall k \in \mathcal{K}. \quad (3)$$

Accordingly, the total compressed payload size of task k is

$$B_k = N_k \sum_{q \in \mathcal{D}} b_q x_{k,q}. \quad (4)$$

Pick one depth per task, and the task's total size is its image count times whichever per-image payload that depth costs – the accounting every later constraint and objective term builds on.

Let $r_k(t)$ denote the number of images of task k fully delivered to its destination GBS by the end of slot t . The delivery fraction of task k is

$$\rho_k(t) = \frac{r_k(t)}{N_k}, \quad 0 \leq \rho_k(t) \leq 1. \quad (5)$$

Let $y_{k,ij}(t) \geq 0$ denote the number of bits of task k transmitted over link (i, j) during slot t . The link-capacity constraint is

$$\sum_{k \in \mathcal{K}} y_{k,ij}(t) \leq C_{ij}(t) \Delta t, \quad \forall (i, j) \in \mathcal{E}_t, \quad (6)$$

i.e. every task sharing a link has to fit inside that link's capacity for the slot, together.

Let $Q_{i,k}(t)$ denote the remaining amount of task k 's data stored at node i at slot t . The queue state evolves according to arriving compressed data and transmitted traffic, subject to standard flow conservation over the time-varying network: data leaves a queue only by being transmitted along a feasible link, and can't exceed what has actually arrived at that node so far.

To capture latency sensitivity, let $\phi_k(t) \in [0, 1]$ denote the freshness weight of task k at slot t :

$$\phi_k(t) = \begin{cases} 1, & t\Delta t \leq d_k, \\ \exp(-\alpha_k(t\Delta t - d_k)), & t\Delta t > d_k, \end{cases} \quad (7)$$

where α_k controls how quickly value decays after the deadline ($\alpha = 1/300 \text{ s}^{-1}$ in our experiments). Delivered before the deadline, an image counts at full value; delivered late, it's still worth something but decays exponentially – a wildfire photo a day late is close to worthless, one five minutes late is nearly as good as on time. Tasks $|\mathcal{K}| = 64$ arrive from 8 areas of interest with $N_k \sim \mathcal{U}\{100,000, 300,000\}$ and $w_k \sim \mathcal{U}\{1, 2, 3\}$ (seed 42); a task may be explicitly **dropped** (abandoned mid-flight) or **rejected** (refused at admission), both first-class reported outcomes rather than silent non-delivery.

2.3 Unified utility

Utility used to be defined in five places that disagreed: the reported score, the flat MPC's own objective, the two levels of the hierarchical scheduler, and the offline bound each used a different combination of terms. We use one function for all five. With normalized weight $\bar{w}_k = w_k / \max_j w_j$ and realized arrival time $t^{\text{arr}} = (t+1)\Delta t + \delta_{k,t}$ along the path actually used for delivery,

$$U_k = \bar{w}_k \left[\omega_Q s_{q_k} \hat{G}_k - \omega_T T_k - \omega_E C_k \right], \quad (8)$$

$$U = \sum_k U_k + \omega_F |\mathcal{K}| \min_k \hat{U}_k, \quad (9)$$

with default weights $\omega_Q = 1.0$, $\omega_T = 0.25$, $\omega_E = 0.05$, $\omega_F = 0.1$. Each task earns reward proportional to how much of it got delivered, scaled by compression quality (s_{q_k}) and delivered fraction ($\hat{G}_k$), minus a lateness penalty (T_k) and a resource-cost penalty (C_k); the system-wide score sums every task's net reward and adds a bonus tied to whichever task did worst, so nothing can rack up a high score by quietly abandoning one inconvenient task. Here $\hat{G}_k \in [0, 1]$ is a concave coverage gain over delivered, freshness-weighted image fraction (§2.4); tardiness is

$$T_k = \sum_t \frac{\Delta r_{k,t}}{N_k} \min\left(1, \frac{\max(0, t^{\text{arr}} - d_k)}{\Delta_{\text{ref}}}\right), \quad (10)$$

so each newly-delivered slice of images is penalized in proportion to how late it arrived past the deadline, capped at 1; resource cost is

$$C_k = \sum_t \frac{\Delta r_{k,t}}{N_k} \left[\hat{\omega}_{\text{tx}} \frac{b_q}{b_{\text{max}}} + \hat{\omega}_{\text{enc}} \right], \quad (11)$$

a normalized charge for bandwidth and encode effort spent on each delivered slice; and $\hat{U}_k = U_k / (\omega_Q s_{\text{max}} \bar{w}_k) \leq 1$ is a scale-free, weight-relative normalization of U_k used only inside the fairness term, so a naturally small task (low weight) doesn't get unfairly flagged as the worst-off one just because its raw U_k is small.

Legacy identity. Set $\omega_Q=1$, $\omega_T=\omega_E=\omega_F=0$, one coverage segment of unit width/slope, no weight normalization, and $s_q = 1 - \text{LPIPS}_q$, and this collapses exactly to the original score $\sum_k \sum_t w_k \phi_k(t^{\text{arr}}) u_{qk} \Delta r_{k,t}/N_k$ – legacy scoring is a parameter setting of this same function, not a separate code path, and every previously-committed legacy number reproduces exactly under this setting.

2.4 Concave coverage without integer variables

Getting the first fraction of a task’s images delivered is worth more, per image, than getting the last fraction – a concave, diminishing-returns reward. Encoding a curved reward inside a linear program normally needs extra machinery (binary or SOS2 variables, expensive to solve). Instead, coverage gain is linearized over $J=4$ segments of width Δ_j and strictly decreasing slopes $m_1 > \dots > m_J$ with $\sum_j \Delta_j m_j = 1$ (default: widths (0.25, 0.25, 0.25, 0.25), slopes (2.00, 1.12, 0.60, 0.28), a discretization of $g(\rho) = (1 - e^{-3\rho})/(1 - e^{-3})$):

$$\hat{G}_k = \sum_{\tau,j} m_j \phi_k(t_\tau^{\text{arr}}) \lambda_{k,q,\tau,j}, \quad (12)$$

$$\sum_j \lambda_{k,q,\tau,j} = \frac{y_{k,q,\tau}}{N_k}, \quad \sum_{q,\tau} \lambda_{k,q,\tau,j} \leq \Delta_j^{\text{res}}, \quad (13)$$

where Δ_j^{res} is the width of segment j still unclaimed by coverage already realized before the current planning horizon (skip this residual and the MPC would re-earn steep early-segment credit at every re-plan, double-counting value already banked). Picture each task’s 0–100% delivered range as four stacked buckets, each paying less per percentage point than the one before; $\lambda_{k,q,\tau,j}$ is how much of bucket j a given slot’s delivery fills. Because the payoff always shrinks bucket by bucket, a solver that only understands straight lines will still fill bucket 1 before touching bucket 2, entirely on its own – no binary “which bucket am I in” variable needed, and the linear relaxation of this objective is exact.

2.5 Fairness: maximin, not Jain

Jain’s index $(\sum_k U_k)^2/(|\mathcal{K}| \sum_k U_k^2)$ is a classic networking fairness statistic, but it’s a ratio of quadratics and can’t be represented inside a mixed-integer linear program, so it’s reported only as a diagnostic, never optimized directly. What’s actually optimized is a maximin floor – how well the single worst-off task did – via one continuous column $u_{\min}$ and $|\mathcal{K}|$ constraint rows $u_{\min} \leq \hat{U}_k$, entering the objective as $\omega_F |\mathcal{K}| u_{\min}$ (the second term of U above). $u_{\min}$ needs a free, unbounded-below lower bound, since $\hat{U}_k$ can go negative once tardiness/cost penalties exceed the quality gain for a badly-served task.

2.6 Flat MPC objective

At each decision slot t , the scheduler observes the current network state – task queues, remaining deadlines, satellite positions, available links – and predicts network evolution over a finite horizon of H slots: $\hat{\mathcal{E}}_{\tau|t}$, $\hat{C}_{ij}(\tau|t)$, $\hat{Q}_{i,k}(\tau|t)$ for $\tau = t, \dots, t+H-1$. It jointly determines compression depth, transmission timing, and traffic allocation over that horizon, then executes only the first-slot decisions; at slot $t+1$ it observes the new state, shifts the horizon forward, and solves again. The rolling-horizon MILP solved by the flat mpc scheduler (re-solved every 5 slots or on new-task arrival; $H = 60$ slots) is

$$\max \sum_{k,q,\tau} \left[w_k \phi_k(t_{k,\tau}^{\text{arr}}) u_q + \lambda_1(H-\tau) b_q \cdot 8/10^9 - \lambda_{\text{late}} w_k \frac{\max(0, t_{k,\tau}^{\text{arr}} - d_k)}{\Delta_{\text{ref}}} \right] \frac{y_{k,q,\tau}}{N_k}, \quad (14)$$

with $\lambda_{\text{late}} = 5 \times 10^{-2}$, $\Delta_{\text{ref}} = 300\text{s}$, subject to: one depth per task per slot, link/GBS capacity, the encoder throughput pipeline (81 images/s per satellite), and cumulative delivered-volume consistency. Here $t_{k,\tau}^{\text{arr}} = (t + \tau + 1)\Delta t + \text{route delay}$ uses the real propagation delay of the path actually

assigned to (k, τ) . The first bracket term rewards weighted, freshness-discounted useful delivery; the second gives a small forward-looking bonus to plans that keep transmitting rather than idle capacity; the third subtracts an explicit lateness penalty – which lets the MPC rationally choose to stop pushing a task’s backlog once its cost outweighs its already-decayed value, instead of that decision happening implicitly through silent non-delivery. §2.9 below makes this same tradeoff explicit via admission control and drop.

2.7 Congestion-aware predictive routing

The default router picks the geometrically shortest path regardless of how busy it currently is, which can overload a popular link while a nearly-as-fast alternative sits idle. The congestion-aware variant (**mpc-congestion**) instead predicts each link’s cost as propagation delay plus a queueing-theory-style congestion penalty computed against the MPC’s own current plan:

$$w_e(\tau) = d_e(\tau) + \beta d_{\text{ref}} \frac{\rho_e(\tau)}{1 - \min(\rho_e(\tau), \rho_{\text{max}})}, \quad \rho_e(\tau) = \frac{f_e(\tau)}{C_e \Delta t}, \quad (15)$$

where $f_e(\tau)$ is the bits the current plan places on edge e at predicted step τ , and $\rho_e(\tau)$ is that edge’s predicted utilization. A link’s effective cost stays close to its physical delay while mostly empty, then rises sharply as it approaches saturation ($\rho_e \rightarrow \rho_{\text{max}}$), which spreads traffic across several good paths instead of jamming everyone onto the one “best on paper” route. Because the plan depends on routes and routes depend on the plan, this is iterated to a damped fixed point (method-of-successive-averages, $\eta_{\text{it}} = 1/(\text{it}+2)$, 3 rounds, keep-best-iterate), with ρ clipped at 0.99 so a saturated link is never treated as disconnected. This coupling is only measurable under the fabric-limited capacity regime (§3) – under the gbs-limited regime the ground-station aggregate budget dominates every route regardless of ISL cost, and congestion-aware routing is a provable, verified no-op there.

2.8 Two-MPC couplings: routing and depth

mpc-2level: a dedicated routing MPC solves a min-cost multicommodity-flow linear program over candidate paths,

$$\min \sum_{k,p,\tau} f_{k,p,\tau} \frac{\delta_{p,\tau}}{\Delta_{\text{ref}}^{\text{delay}}} + M \sum_{k,\tau} \text{sh}_{k,\tau} \quad \text{s.t.} \quad \sum_p f_{k,p,\tau} + \text{sh}_{k,\tau} = D_{k,\tau}, \quad \sum_k \sum_{p \ni e} f_{k,p,\tau} \leq C_e \Delta t, \quad (16)$$

where $f_{k,p,\tau}$ is the flow assigned to task k on candidate path p , $\text{sh}_{k,\tau}$ is an explicit “shortfall” variable absorbing any demand no path can carry (penalized heavily by M so it’s only used when genuinely unavoidable), and $D_{k,\tau}$ is task k ’s desired bit demand at step τ . This LP spreads each task’s traffic across whichever candidate paths have room, at minimum total delay, instead of committing everything to one path – and if no combination of paths can carry all of it, the shortfall variable makes that gap explicit rather than the LP going infeasible. This routing LP and the depth MILP (§2.6) exchange demand $D_{k,\tau}$ and per-edge flow shares, iterated to a fixed point with the same successive-averages damping as §2.7. Critically, $D_{k,\tau}$ has to be the task’s *desired* bits, not what the depth MILP already conceded – deriving it from the previous plan would be circular and silently reproduce flat MPC even under real congestion (a correctness trap caught during development). Iteration 0 is defined to be exactly flat **mpc**, so **mpc-2level** can never plan worse than flat MPC plans, by construction.

mpc-hier-route: a route-coordinator MPC runs once per 10-minute macro-epoch over a 60-slot horizon in 10-slot macro-windows (topology evaluated at each window’s middle slot, as a forecast), solving the same flow LP to freeze the top- K candidate paths per task for the whole epoch; the fast depth MILP then solves only within those frozen paths. Empty route assignments reproduce **mpc-hier** (§2.9) exactly, verified bit-for-bit.

Measured: route freezing does not survive a LEO fabric. A route frozen at epoch start is usable at its own executed slot only 14.4% of the time in general (our fabric-limited scenario:

7.7%), falling to 0% by a 570-second lookahead – because ground-station contact windows average only 227–265 s, shorter than any planning epoch worth having. That’s a structural property of the constellation geometry, not a tuning failure: it’s a negative result about freezing *concrete paths* specifically, not necessarily about freezing a more abstract routing decision (e.g. a serving-GBS assignment), which remains untested.

2.9 Hierarchical MPC and the rotting-backlog fix

mpc-hier pairs a slow upper-level coordinator (cadence 20 slots / 10 min, horizon 6 macro-steps ≈ 2 h, an LP over continuous relaxed depth $x_{k,q} \in [0, 1]$ and GBS-aggregate budget only) with a fast lower-level per-task MILP (cadence 5 slots or on arrival, horizon 20 slots, integral depth $x_{k,q}$, the same machinery as §2.6). The upper level decides, at a coarse cadence, which tasks get admitted and roughly how much budget each gets; the lower level decides, at a fine cadence, the exact depth and schedule within that budget – a smaller, faster-solved problem in exchange for real admission/drop activity. Three mechanisms address a specific failure mode, stale-backlog starvation (a task whose freshness has decayed to near zero but still generates optimization variables and is never served nor removed): admission control at arrival, explicit drop once freshness falls below a threshold or a task is sufficiently overdue (reported as **dropped**, not hidden), and a backpressure aging term in the lower-level objective that raises priority the longer a task has waited. These thresholds are first-pass tuned (five combinations tried), not an exhaustive search.

mpc-hier vs. the routing/depth split. **mpc-hier**’s two levels both operate on depth (a relaxed depth budget above, an integral depth choice below), with static routing at both levels – a different pairing from **mpc-2level/mpc-hier-route**, whose split is one MPC dedicated to routing and one to depth. [CONFIRM WITH XUANHAO: Xuanhao’s email asked to drop “the previous mpc-hier results where the two MPCs are both used for depth selection.” That description textually matches **mpc-hier** as just defined, which makes it the likely candidate for what Xuanhao means – but this is still pending his direct confirmation of scope.]

2.10 Offline optimality bound

To know whether a scheduler’s score is actually good, it has to be compared to the best *possible* score under the same rules – an offline bound, computed three ways of increasing tightness: (1) an analytic ceiling (every image at the best depth, delivered instantly – a sanity check only, not a meaningful bound); (2) a linear-programming bound (depth relaxed to continuous $x_{k,q} \in [0, 1]$, time aggregated into 5-slot windows, provably-non-binding link/GBS rows dropped, utility evaluated at each window’s *start* to remain a genuine upper bound – never optimistic about when delivery actually happens); and (3) a mixed-integer bound (the same relaxed constraints at full time resolution with integral depth, reporting the solver’s dual bound even when the search hasn’t fully converged within its time budget, which remains a valid upper bound regardless). The governing correctness rule: every term in a scheduler’s realized score must also appear in the bound’s objective, relaxed only in the optimistic direction – enforced at runtime by an assertion that the bound is never smaller than any scheduler’s realized score.

3 Experimental Setup

This section specifies every parameter needed to reproduce the results in this paper from the same simulator (`hypatia_sim/oec_sim/`, built on the Hypatia LEO orbital-mechanics/topology engine).

3.1 Constellation and ground segment

Table 1 lists the physical scenario parameters (`hypatia_sim/oec_sim/config.py`). Four named constellations are implemented (`kuiper-630`, `starlink-550`, `telesat-1015`, `small-walker`); all results in this paper use `kuiper-630` unless stated otherwise.

Table 1: Constellation, link, and task-load parameters.

Parameter	Value
Constellation	Kuiper-630 shell 1: Walker-delta 1156/34/1 (34 planes $\times$ 34 sats)
Altitude / inclination	630 km / 51.9°
Orbital period	5,830 s
Ground stations G	Tokyo, New York, São Paulo, Sydney (4 total), min. elevation 20°
ISL topology	+Grid, 2,312 candidate links; feasible if graze altitude > 80 km above Earth’s surface <i>and</i> range $\leq$ 5,016 km
Areas of interest	8 (California-fires, Amazon-basin, Sahel, Ganges-delta, Barrier-reef, Dnipro-basin, and 2 more; see <code>config.py:A0IS</code>)
Tasks per run $ K $	64, one per (AOI, publication event), Poisson-triggered
Images per task N_k	$\sim \mathcal{U}(100,000, 300,000)$, total $\approx$ 12.81M images
Task weight w_k	$\in \{1, 2, 3\}$
Random seed	42 (task generation; sensitivity sweeps use 5 seeds)
Simulation horizon	18,000 s (5 h, $\approx$ 3.1 orbits)
Slot length Δt	30 s $\Rightarrow$ 601 decision slots
MPC planning horizon H	60 slots (30 min look-ahead)

3.2 Link capacity regimes

Two named capacity regimes share the identical topology and task model but rebalance link rates to make different resources binding (Table 2). **gbs-limited** is the default; its per-GBS aggregate budget is $\geq 12\times$ tighter than any ISL segment a route could cross, so ISL contention is mathematically unreachable there. **fabric-limited** throttles the ISL rate specifically so routing decisions have a measurable effect. Both regimes are used across the scheduler-comparison and decision-distribution results this setup feeds, written up separately.

Table 2: Link capacity regimes (`config.py:SCENARIOS`).

Regime	ISL rate	GSL rate (per sat-GBS pair)	GBS aggregate rate
gbs-limited (default)	100 Mbps	100 Mbps	2 Mbps
fabric-limited	1 Mbps	50 Mbps	500 Mbps

3.3 Compression depths and payload

Each task-image is encoded on board at one of $D = \{2, 4, 8, 16\}$ RQ-VAE residual-quantization depths before downlink. Payload scales linearly with depth, $b_q = 88q$ bytes (measured on the FLAIR-1 8×8 dedicated models): $b_2 = 176$ B, $b_4 = 352$ B, $b_8 = 704$ B, $b_{16} = 1,408$ B. On-board encode time is $12.34 \text{ ms} \pm 0.03\text{--}0.05 \text{ ms}$ per image and is *depth-independent* (measured identically at $8\times 8\times 1$ and $8\times 8\times 8$), giving an encoder throughput of 81 images/s per satellite regardless of the depth chosen.

3.4 Decision cadence: what “every 30 s” actually means

The scheduling slot is $\Delta t = 30$ s, but the MPC does *not* re-solve its optimization every slot. Each solve plans $H = 60$ slots (30 min) ahead and commits only to the first slot of that plan; the scheduler re-solves only when either 5 slots have elapsed since the last solve, or a new task arrives — otherwise it executes the next committed step of the existing plan. Over the full 601-slot (5-hour) simulation, the flat mpc scheduler solves the optimization approximately 121 times, not 601 times. **Reported solve-time statistics (mean/p95/max) are for a single individual solve** (how long the optimizer takes to think once); a separately reported *total* solve-time figure

is the sum of individual solve times across the whole run — total compute time spent solving, not wall-clock simulation time and not a per-slot cost.

3.5 Software and solver

The scheduling MILPs/LPs (flat MPC, hierarchical admission/budget LP, the two-level routing LP, and the offline oracle bound) are solved with HiGHS via `scipy.optimize.linprog/milp`. Orbital mechanics, contact windows, and link feasibility come from Hypatia’s satellite-generation and topology modules (`hypatia/satgenpy`), vendored unmodified.

3.6 Quality-table provenance

The utility function’s quality term s_q (§2) is grounded in measured downstream FLAIR-1 segmentation mIoU, source label `flair-unet-r34-rgb1e / val7050 / 2026-08-31` (full 7,050-image validation population; the measurement itself is written up separately). Every scheduler result this setup feeds uses this real, measured table (`hypatia_sim/oec_sim/quality_table.json`), not the placeholder fallback (`config.QUALITY_TABLE_FALLBACK`) that the code falls back to when this file is absent.